
DYNAMIC QUERY MODIFICATION FOR BINARY LOCALITY SENSITIVE HASHING

PREPRINT

Ben Claydon, Richard Connor, Alan Dearle

University of St Andrews

St Andrews, Scotland

{bc89,rchc,al}@st-andrews.ac.uk

May 2026

ABSTRACT

Our context of interest is how binary locality sensitive hash (LSH) functions can be used to solve the approximate near neighbour (ANN) problem in hyperspherical spaces, which seeks to find the k closest elements of some dataset $X \subset \mathbb{R}^d$ to some further point $\mathbf{q} \in \mathbb{R}^d$ presented as a query. Binary locality sensitive function families $\mathcal{H}$ are sets of functions each with signature $h : \mathbf{x} \in \mathbb{R}^d \rightarrow \{0, 1\}$. A function is locality sensitive if and only if the output of the function is more likely to be equal (a ‘hash collision’) if two close vectors are used as input than if two far vectors are used.

A data structure can be built by generating binary hash codes for each member of X , which are generated by drawing and applying one or more functions from $\mathcal{H}$. When $\mathbf{q}$ is presented as a query, the same set of functions are applied to it and those elements of X with equal binary hash codes retrieved.

In this paper we introduce *dynamic query modification*. This procedure changes $\mathbf{q}$ at query time using information acquired during search to approximate the point $\langle \mathbf{c} \rangle$, which is the ℓ_2 -normalised centroid of the k -nearest neighbour set of $\mathbf{q}$. By theoretical and experimental analysis we demonstrate $\langle \mathbf{c} \rangle$ has two significant advantages when compared to $\mathbf{q}$. Firstly, we show that the hash function output of $\langle \mathbf{c} \rangle$ is likely to have a higher average collision probability with the k -nearest neighbours than the hash output of $\mathbf{q}$. Secondly, in our experiments, we observe no instances of $\langle \mathbf{c} \rangle$ failing to collide with any member of the near-neighbour set; a property which we demonstrate is not true for $\mathbf{q}$.

To demonstrate the efficacy of the technique, we define a novel structure MQ-Forest, a modified version of RP-Forest. Both are binary LSH-based ANN mechanisms, but MQ-Forest dynamically estimates $\langle \mathbf{c} \rangle$ during the query process. We show that MQ-Forest reduces both build and query times by up to 40% when measured over several large, high-dimensional benchmark datasets.

1 Introduction

Searching collections of large, high-dimensional data is an increasingly important task in modern applications. Our context of interest is finding an approximation of the k closest elements of some dataset $X \subset \mathbb{R}^d$ to some further point $\mathbf{q} \in \mathbb{R}^d$ presented as a query. We exclusively consider hyperspherical spaces, which are those where all vectors have an ℓ_2 -norm of 1. Finding the exact k nearest neighbours to $\mathbf{q}$ in high-dimensional datasets is known to require a similarity comparison between $\mathbf{q}$ and each member of X due to the ‘curse of dimensionality’. However, scalable solutions to such queries are required by many applications. Approximate near neighbour (ANN) solutions allow search in sublinear time.

One technique used by algorithms in this category is locality sensitive hashing (LSH). Binary LSH functions accept a point $\mathbf{x} \in \mathbb{R}^d$ as input, and return a binary output value. They possess the property that the output of the function when applied to two near points is more likely to be equal (a ‘hash collision’) than when the function is applied to two far points.

It may seem intuitive that the query sits at the centre of $knn(\mathbf{q})$. We show that this is not typically the case which introduces significant pathologies for LSH-based mechanisms. This suggests the surprising possibility that there exists an input to LSH functions which more effectively retrieves $knn(\mathbf{q})$.

In this paper, we describe a significant improvement which can be made to LSH-based ANN algorithms based on some novel mathematical observations on high-dimensional spaces. We make the following observations:

- In Section 4, we demonstrate the existence of a set $\Phi \subset \mathbb{R}^d$. Each member of Φ has the property that, when used as input to a locality sensitive hash function, they have an equal or higher probability than $\mathbf{q}$ of hash colliding with near neighbours of $\mathbf{q}$. If an element of Φ can be identified and queried in place of $\mathbf{q}$, the performance of LSH-based ANN algorithms can be improved as near neighbours of $\mathbf{q}$ will be retrieved with a greater frequency.
- In Section 5, we provide a practical and fast method to compute a vector likely to be a member of Φ at query time, which can be used to accelerate the performance of binary LSH-based mechanisms.
- In Section 6, we apply the methods described in Section 5 to a popular binary LSH-based ANN algorithm: RP-Forest. We validate the properties of our query modification technique by contrasting the behaviour of a modified and unmodified version of RP-Forest over large, high-dimensional benchmark datasets. We show that our modification to the algorithm results in reduced build and query times when compared to a baseline implementation.

2 Related Work

2.1 Locality Sensitive Hashing

LSH was introduced by Indyk and Motwani [12] as a method for searching high-dimensional spaces. A family of functions $\mathcal{H}$ is locality sensitive if and only if there exists a $p_1 > p_2$ and $t_1 < t_2$ which satisfy inequalities 1 and 2 for all functions $h \in \mathcal{H}$. A corollary of this property is that for any two points $\mathbf{x}, \mathbf{y}$, the probability $\Pr(h(\mathbf{x}) = h(\mathbf{y}))$ decreases monotonically as $\|\mathbf{x} - \mathbf{y}\|_2$ increases.

$$\Pr(h(\mathbf{x}) = h(\mathbf{y})) \geq p_1 \quad | \quad \|\mathbf{x} - \mathbf{y}\|_2 \leq t_1 \quad (1)$$

$$\Pr(h(\mathbf{x}) = h(\mathbf{y})) \leq p_2 \quad | \quad \|\mathbf{x} - \mathbf{y}\|_2 > t_2 \quad (2)$$

Charikar introduced a binary LSH function for Euclidean spaces using randomly generated hyperplanes which pass through the origin [2]. A hash value of 1 is assigned to a point $\mathbf{x}$ if it is above the hyperplane, or 0 otherwise. This LSH function family is almost identical to the node subdivision procedure found in Random Projection Forests (see Section 2.2); the latter only differs from Charikar’s original formulation by shifting the hyperplane a random amount from the origin.

2.1.1 LSH Approximate Near Neighbour Algorithms

LSH functions are often used to create data structures which solve the ANN problem. Locality-sensitivity based ANN algorithms can be viewed abstractly in a manner described by Lv [18]:

The basic LSH indexing method processes a similarity search, for a given query $\mathbf{q}$, in two steps. The first step is to generate a candidate set by the union of all buckets that query $\mathbf{q}$ is hashed to. The second step ranks the objects in the candidate set according to their distances to query object $\mathbf{q}$, and then returns the top K objects.

...

By concatenating multiple LSH functions, the collision probability of far away objects becomes very small (p_2), but it also reduces the collision probability of nearby objects (p_1). As a result, multiple hash tables are needed in order to find most of the nearby objects.

2.2 RP-Trees and RP-Forests

The RP-Tree data structure was first introduced by Dasgupta and Freund as a variant of the KD-Tree suitable for searching high-dimensional spaces [7]. To build an RP-Tree, a node is initialised which forms the root of the tree, and all points in a dataset $X \subset \mathbb{R}^d$ are assigned to this node. Thereafter, a tree is constructed by repeatedly subdividing any node containing a number of points greater than some user-defined parameter n_s into two child nodes.

To subdivide a node, any point $\mathbf{x}$ contained in the node such that $\mathbf{w} \cdot \mathbf{x} > a$, for some plane $\mathbf{w}$ and some real value a , are assigned to the left child, and all other points to the right child. The value a is uniformly randomly generated with upper and lower bounds equal to the greatest height above and below $\mathbf{w}$ of any point contained in the node. When $a = 0$, this hash function is analogous to Charikar’s hyperplane LSH.

The query procedure of an RP-Tree begins at the root node, where the predicate $\mathbf{w} \cdot \mathbf{q} > a$ is computed using the same instance of $\mathbf{w}$ and a which split the root node. If this predicate is true, then the left subtree is recursively searched, or the right subtree is recursively searched if not. This process continues until a leaf node is encountered, and the objects contained therein are retrieved as a ‘candidate set’. Only points in the candidate set will be measured for similarity to $\mathbf{q}$, and the k closest returned as the approximate near neighbour set.

As this paper discusses two separate families of binary LSH function, we refer to $\mathcal{H}_{rp}$ as the LSH schema used by RP-Forest, and $\mathcal{H}_c$ as the LSH schema defined by Charikar. We use $\mathcal{H}$ to refer to either function family.

RP-Forest is an ensemble of several independently constructed RP-Trees. When a query is presented, each RP-Tree is searched, and a candidate set is formed by taking the union of all objects retrieved from the trees. We use Yan et al.’s formulation of RP-Forest, laid out in Algorithm 1 of their study [28]. Applications include studies on gene data [9] [29], pose estimation [16] [17], and outlier detection [25].

2.3 Query Modification

Here we review methods of modifying a user-provided query vector to permit the accelerated retrieval of relevant elements from a large collection, which is also the aim of the novel query modification algorithm we present.

2.3.1 Multiprobe LSH

There are two forms of multiprobe LSH: entropy-based [18] and those which are dependent on LSH bin geometry such as the schema introduced by Andoni et al. for the cross-polytope LSH family [1]. We will focus on the former, as it is a technique which relies on truly modifying the query vector rather than an additional step in an LSH-based query mechanism.

Entropy-based multiprobe is the process of perturbing a query vector by a random Gaussian vector in order to change its hash value. If the degree of perturbation is scaled appropriately, then a new vector is generated which is similar to the original query but hashes to a different value. Additional candidates may be retrieved from this new hash bin, as it is also likely to contain items which are similar to $\mathbf{q}$. In this scenario a new hash table need not be searched to find additional relevant objects, which reduces the amount of space and time required by the LSH-based ANN algorithm.

The methods we present also rely on modifying the query to raise the likelihood of retrieving similar objects to the query by identifying and using a point in Φ . However, we exploit information gained during a search procedure, as opposed to an entirely random perturbation. In this way, our query modification method is complementary to multiprobe LSH as both techniques can be used simultaneously.

2.3.2 Relevance Feedback

The idea of modifying a query to permit the retrieval of more relevant objects from a large collection is well established as ‘relevance feedback’ in the domain of information retrieval. Relevance feedback attempts to solve the problem of different users of an information retrieval (IR) system having different needs. Results retrieved by an IR system may be relevant to some users, but not others depending on the requirements of their query.

To solve this problem, users are asked to provide relevance feedback on each member of the set of results retrieved by the IR system. This feedback can take the form of the binary label ‘relevant’ or ‘not relevant’, or some numerical score of relevance [30]. Upon receiving this feedback, the IR system will formulate a new query, and perform another search using it. This new query should somehow be more similar to the ‘relevant’ documents, and dissimilar to the ‘irrelevant’ documents.

One classical example of a method to modify a query vector using relevance feedback is Rocchio’s algorithm [24]. This algorithm modifies the query vector in the following way:

$$\mathbf{u} = a\mathbf{q} + b \frac{1}{D_r} \sum_{\mathbf{r} \in D_r} \mathbf{r} - c \frac{1}{D_{nr}} \sum_{\mathbf{n} \in D_{nr}} \mathbf{n} \quad (3)$$

where a, b, c are some weights, D_r is the set of relevant items as judged by the user, and D_{nr} are the irrelevant items. Rocchio’s algorithm is conceptually simple: it moves the query vector closer to the centroid of the set of relevant objects, and away from the centroid of irrelevant objects.

Rocchio’s algorithm is applied iteratively by generating several sets of results and incorporating new user feedback for each. Examples of systems which use relevance feedback in this way are [26] and [5]. This iterative process will result in a query which is personalised to the user’s specific requirements.

2.3.3 Pseudo-Relevance Feedback

Relevance feedback assumes that there is a human in the loop, who guides the query as it progresses. Some IR systems utilise *pseudo-relevance feedback* (PRF), which assumes the most similar documents to the query are ‘relevant’ and others are ‘not relevant’. This technique permits the refining of queries automatically, without additional human input.

Examples of use of this technique include local context analysis [27], which is a method of applying PRF to document retrieval. After results are retrieved by the IR system, local context analysis automatically forms a new ‘expanded’ query by selecting new keywords based on those which frequently occur in retrieved documents. An expanded query is a query which has been modified to include additional relevant information. Results show that this expanded query retrieves more relevant documents than the original.

This technique can also be used in dense vector spaces, such as the ones we study. Kuzi et al. studied the retrieval of documents using expanded queries [15]. Instead of an iterative method, queries were expanded using similar words as judged by a Word2Vec [19] word embedding model trained on the document corpus. This form of PRF was also shown to produce a query which improves retrieval of relevant documents.

Faggioli et al [8] presented a learning-based approach to form an expanded query by selecting some small number of important dimensions of the original query embedding, and setting the rest to 0. By modifying the query in this way, the authors showed that more relevant documents are retrieved than when the original query is used. Furthermore, query expansion methods for LSH algorithms have been attempted [6] [10] [14] with learning-based approaches.

3 Properties of Generic LSH Mechanisms

Any family of locality sensitive functions must obey Inequalities 1 and 2. In the context of the ANN problem, the property of interest is *what fraction of $knn(\mathbf{q})$ collide with $\mathbf{q}$* . However, the properties of an LS function family make no such guarantees in the latter context.

In previous work, we have shown how this lack of guarantee can hinder binary LSH-based ANN mechanisms significantly [4]. There, we introduced the notion of a ‘hash failure’, and demonstrated they occur frequently. A hash failure occurs when $\mathbf{q}$ hash collides with only a small fraction of its near neighbours. A hash failure can occur because, even if the probability of hash collision between $\mathbf{q}$ and each of its near neighbours is individually high, these probabilities may be *non-independent* and therefore strongly correlated since independence is not a property guaranteed by an LS function family.

The existence of such failure cases particularly impacts the probability of hash collision between *compound* hash functions. Compound hash functions are formed from the combination of independently drawn hash functions. In the context of the binary hash functions we study, a compound hash function generates a single binary string by concatenating the output of multiple independent binary LSH functions. A hash collision between two compound hash functions occurs when these binary strings are equal.

A compound hash function collides with a number of elements of the k nearest neighbour set proportionally to the *product* of the recall of each constituent binary hash function. Therefore, the failure of *any* individual hash function is not tolerable.

Figure 1a shows the distribution of $knn(\mathbf{q})$ collision rates after a large number of binary hash functions are applied to a dataset of image embeddings, with random objects from the dataset selected to use as queries. As can be seen, many functions retrieve a very small fraction of the near neighbours. Figure 1b shows the same observations, where an element $\langle c \rangle \in \Phi$ is used in place of $\mathbf{q}$ ¹. A fuller explanation of this experiment, and results on other datasets, are provided in Section 6.4. Unlike $\mathbf{q}$, we empirically observe no instances of hash failure when this point is used as input to LSH functions.

¹We will demonstrate how to derive $\langle c \rangle$ and Φ in Section 4.1.

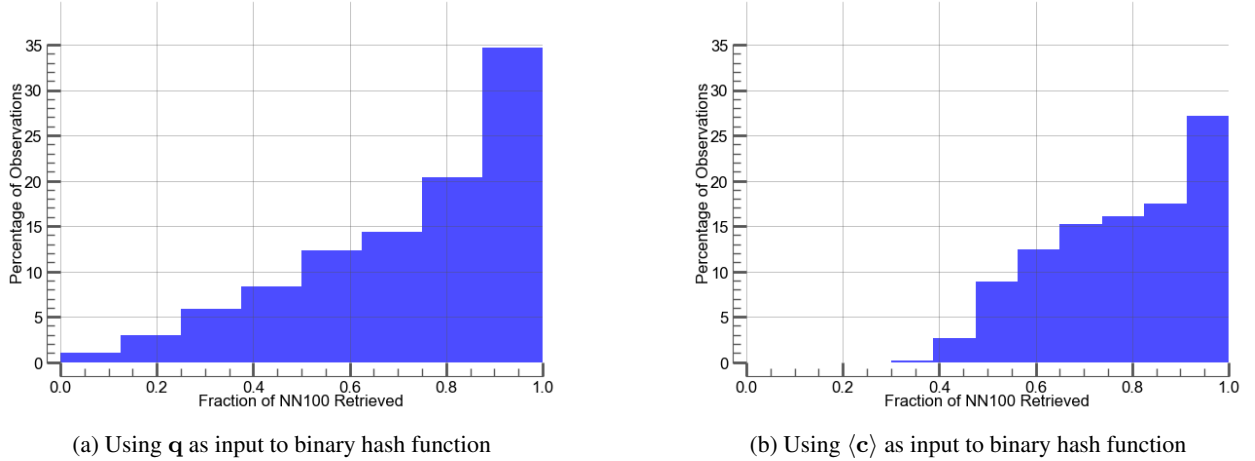


Figure 1: Fraction of points in $knn(\mathbf{q})$ which hash collide with $\mathbf{q}$ and $\langle \mathbf{c} \rangle$ respectively. Measured by applying 1000 random functions from $\mathcal{H}_{rp}$ to 100 random queries, over the MirFlickr Dino2 dataset. It is possible for $\mathbf{q}$ to collide with only a small fraction of $knn(\mathbf{q})$, whereas this is not observed for an element $\langle \mathbf{c} \rangle \in \Phi$.

4 Query Modification

4.1 Defining Modified Query Quality

We define the set Φ as those elements of $\mathbb{R}^d$ whose hashes are equally or more likely to collide with $knn(\mathbf{q})$ than $\mathbf{q}$. We define a measure of this likelihood as the *average collision probability* between some point $\mathbf{u} \in \mathbb{R}^d$ and each member of $knn(\mathbf{q})$:

$$ACP(\mathbf{u}, knn(\mathbf{q})) = \frac{1}{k} \sum_{i=0}^{k-1} \Pr_{h \sim \mathcal{H}} [h(\mathbf{u}) = h(\mathbf{x}_i)], \quad \mathbf{x}_i \in knn(\mathbf{q}). \quad (4)$$

We more formally define the set Φ to be those points where $ACP(\phi \in \mathbb{R}^d \mid \|\phi\|_2 = 1) \geq ACP(\mathbf{q})$. In Appendix B we describe a statistical model which computes the ACP function for the family $\mathcal{H}_{rp}$, for arbitrary values of $\mathbf{u}$ and $knn(\mathbf{q})$. Furthermore, in Section B.1, we also show that this model can be used to compute the ACPs for the family $\mathcal{H}_c$ as well.

We further define a member $\langle \mathbf{c} \rangle \in \Phi$ which has, to a first-order approximation, the highest ACP of any member of Φ . In Appendix C, we demonstrate the ℓ_2 -normalised geometric centroid of the set $knn(\mathbf{q})$ meets this condition. Moreover, this point is extremely cheap to compute given an arbitrary set $knn(\mathbf{q})$, as we will go on to show.

4.1.1 ‘Hash Failure’ Cases

Maximising average collision probability is necessary but not sufficient to ensure good search outcomes. Hash failure cases must also be rare to ensure the reliable performance of compound hash functions. To prevent hash failure, we require a lack of correlation among outputs of a single hash function as applied to each element of $knn(\mathbf{q})$. We refer to this as an *average correlation* value. This is because hash failures occur when the output of a single function is equal when applied to most members of $knn(\mathbf{q})$ due to this correlation, and $\mathbf{q}$ hashes to a different value.

In Appendix C.1, we provide a theoretical explanation of why hash failures occur, as well that hash failure cases are empirically eliminated in our experiments when $\langle \mathbf{c} \rangle$ is used as the input to hash functions. We show that the average correlation depends on only three geometric properties: the inter-point distances between members of $knn(\mathbf{q})$, the distance between $\mathbf{u}$ and each member of $knn(\mathbf{q})$, and the height of $\mathbf{u}$ above $\mathbf{w}$. As the inter-point distances between members of $knn(\mathbf{q})$ are a fixed quantity, and the height of $\mathbf{u}$ above $\mathbf{w}$ cannot be controlled as $\mathbf{w}$ is randomly generated, we may only influence the average correlation by changing the position of $\mathbf{u}$. We demonstrate that the average covariance between near neighbour heights above / below the hyperplane $\mathbf{w}$ is minimal when $\mathbf{u} = \langle \mathbf{c} \rangle$, thereby reducing the probability of most or all near neighbours being above or below $\mathbf{w}$. Furthermore, in Appendix D, we experimentally verify that this value is approximately 0 when measured over benchmark datasets.

5 Description of LSH Algorithm Modification

5.1 Overview

In Section 5.2, we describe a modification which can be made to binary LSH mechanisms which creates, refines, and utilises an estimate of $\langle c \rangle$ at query time. We call such an estimate $\langle \hat{c} \rangle$. In Section 5.3, we discuss the computational cost of computing $\langle \hat{c} \rangle$ and present a method for accelerating it.

5.2 Structure of LSH-Based ANN Algorithms

In the parlance of this paper, LSH-based ANN algorithms can abstractly be described as:

- All LSH-based ANN mechanisms store one or more independent compound hash functions for each member of X . These are constructed at preprocessing time.
- When a query is presented, LSH-based ANN mechanisms apply the same set of compound hash functions to the query. Only those points from X where at least one compound hash collision has occurred are retrieved as a candidate set.
- By increasing the number of compound hash functions stored, the expected recall will increase, at the cost of additional preprocessing and query time.

A flowchart detailing one possible implementation of this process is shown in Figure 2a. This abstract LSH algorithm begins by hashing the query point to create the first compound hash function. A set Z is generated from those members of X which have not been previously retrieved. Elements of Z are re-ranked, and a candidate set is maintained of the k closest to q . This process repeats for each independent compound hash function before returning the candidate set.

Our modification to this class of algorithm is shown by the flowchart in Figure 2b. In outline, our approach is to derive an instance of $\langle \hat{c} \rangle$ from the candidate set after each compound hash function is used to retrieve new objects. This derived instance of $\langle \hat{c} \rangle$ is then used as input to Phase 1 of the next compound hash function to be searched, which in turn returns new unseen points from X from which a new $\langle \hat{c} \rangle$ can be estimated.

We expect this derived $\langle \hat{c} \rangle$ to be a member of Φ with a high probability. Thus, using $\langle \hat{c} \rangle$ as input to the compound hash function instead of q is expected to result in both a greater probability of hash collision with near neighbours of q , as well as a decreased probability of hash failure. For each iteration, the estimate $\langle \hat{c} \rangle$ will become an increasingly accurate estimate of $\langle c \rangle$.

5.3 Cost of Query Update

Naively, the cost of computing the centroid of k points is of $O(k)$ vector operations. As this work must be repeated each time new elements are added to the candidate set, the cost of re-computing $\langle \hat{c} \rangle$ could begin to outweigh any benefit it provides. Thus, we consider how elements of the candidate set may be stored and provide an algorithm which may reduce the cost of computing its centroid.

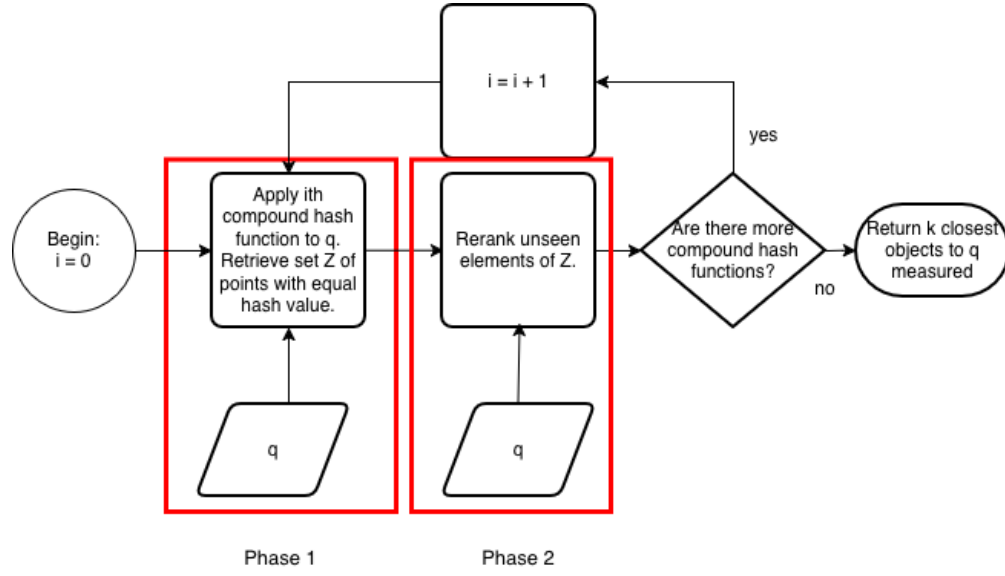
Information on the candidate set can be stored in a priority queue Q where the closest elements to q are given greatest priority. Q may contain at most k elements; if a new element is added and the size of the queue would exceed k , the lowest priority item is removed. When an instance of $\langle \hat{c} \rangle$ is required, the ℓ_2 -normalised centroid of the elements of Q may be computed in $k + 1$ vector operations.

We may reduce this computational cost by storing a further vector s , which stores the value $\sum_{x \in Q} x$. When a point x is added to Q , we set $s = s + x$. If a point y is removed from Q as a result of x being inserted, we set $s = s - y$. To compute $\langle \hat{c} \rangle$, we need only to divide s by its ℓ_2 -norm, which is an extremely fast operation. Algorithm 1 details how Q and s may be updated using a set of points Z , which were retrieved via hashing as in phase 2 of Figure 2b.

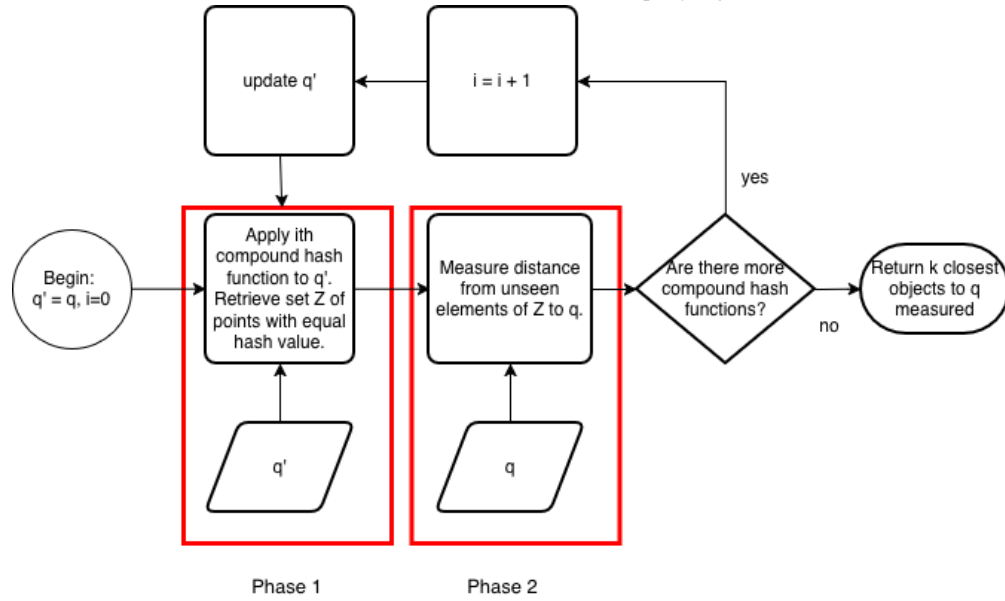
The cost of updating s in this way is proportional to the number of points added and removed from Q . Therefore, Algorithm 1 only saves time if there are fewer than k insertions and removals to Q at any one time. If more than k operations are required, then computing the centroid in the naive fashion becomes the faster option. In Appendix E, we show experimentally that there are often far fewer than k insertions and removals to Q after an RP-Tree is searched, thereby validating the practical applicability of this technique.

6 Experiments

We experimentally verify claims made in the previous sections by performing the following experiments:



(a) Abstract model of an LSH-based ANN query algorithm.



(b) Abstract LSH-based ANN query algorithm which uses query updates.

Figure 2: Comparison of query algorithm for: abstract LSH-based ANN model (above) and our modification (below).

Algorithm 1 Updating $\mathcal{Q}$ and $\mathbf{s}$

```

1:  $\mathbf{z} \leftarrow \min(Z)$  Assign to  $\mathbf{z}$  the point in  $Z$  closest to  $\mathbf{q}$ 
2:  $\mathbf{t} \leftarrow \max(\mathcal{Q})$  Assign to  $\mathbf{t}$  the point furthest from  $\mathbf{q}$  in  $\mathcal{Q}$ 
3: while  $|\mathcal{Q}| < k$  do
4:    $Z \leftarrow Z \setminus \{\mathbf{z}\}$  Remove the closest element from  $Z$ 
5:    $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{\mathbf{z}\}$  Add  $\mathbf{z}$  to  $\mathcal{Q}$ 
6:    $\mathbf{s} \leftarrow \mathbf{s} + \mathbf{z}$  Update  $\mathbf{s}$ 
7:    $\mathbf{z} \leftarrow \min(Z)$ 
8:    $\mathbf{t} \leftarrow \max(\mathcal{Q})$ 
9: end while
10: while  $\|\mathbf{t} - \mathbf{q}\|_2 > \|\mathbf{z} - \mathbf{q}\|_2$  do
11:    $Z \leftarrow Z \setminus \{\mathbf{z}\}$  Remove the closest element to  $\mathbf{q}$  from  $Z$ 
12:    $\mathcal{Q} \leftarrow \mathcal{Q} \setminus \{\mathbf{t}\}$  Remove  $\mathbf{t}$  from  $\mathcal{Q}$ 
13:    $\mathbf{s} \leftarrow \mathbf{s} - \mathbf{t} + \mathbf{z}$  Update  $\mathbf{s}$ 
14:    $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{\mathbf{z}\}$  Add the new closest point
15:    $\mathbf{z} \leftarrow \min(Z)$  Assign to  $\mathbf{z}$  the point in  $Z$  closest to  $\mathbf{q}$ 
16:    $\mathbf{t} \leftarrow \max(\mathcal{Q})$  Assign to  $\mathbf{t}$  the point furthest from  $\mathbf{q}$  in  $\mathcal{Q}$ 
17: end while

```

1. In Section 6.2, we demonstrate the effect of modifying RP-Forest using the techniques outlined in Section 5. We show the efficacy of our technique by measuring decreased build and query time compared to the baseline algorithm.
2. In Section 6.3, we experimentally verify that $\mathbf{q}$ and $\langle \mathbf{c} \rangle$ are often dissimilar.
3. In Section 6.4, we provide evidence that hash failures are essentially eliminated when $\langle \mathbf{c} \rangle$ is used as input to hash functions.
4. In Section 6.5, we measure the degree of approximation the set $\widehat{knn(\mathbf{q})}$ can possess before a useful instance of $\langle \hat{\mathbf{c}} \rangle$ can no longer be derived, and demonstrate that only a coarse estimation of the set $knn(\mathbf{q})$ is required to compute a useful instance of $\langle \hat{\mathbf{c}} \rangle$.

6.1 Datasets Used

This paper studies four large, high-dimensional datasets of differing modalities, which we describe in detail in Appendix A.

6.2 Experimental Comparison of MQ-Forest and RP-Forest

We measure the performance of MQ-Forest, and compare it to the baseline RP-Forest algorithm. The MQ-Forest algorithm operates as follows: the first v trees of the RP and MQ-Forests execute identically (where v is a user-defined parameter). Upon reaching the $v + 1$ th tree, the MQ-Forest uses the procedure outlined in Figure 2b, where the initial instance of $\langle \hat{\mathbf{c}} \rangle$ is generated from the candidate set formed by searching the first v trees. A new estimate of $\langle \hat{\mathbf{c}} \rangle$ is computed after each tree is searched, using the fast centroid computation algorithm as described in Algorithm 1. To measure the performance of each algorithm, we formulate the following experiment:

1. Draw and exclude 250 objects from each dataset for use as queries.
2. Build an RP-Forest and an MQ-Forest over the dataset, with a number of trees ranging from 8 to 96. We fix $n_s = 500$ and $v = 8$. To make a direct comparison, we use exactly the same trees and associated hash functions for both the RP and MQ-Forest.
3. We use each object reserved as a query as input to both the RP and MQ-Forest. For each query, we record the recall for $k = 100$, and the number of distance computations required to serve the query. Note that we consider each vector addition as required by Algorithm 1 as a distance computation to ensure that the cost of centroid computation is taken into account.

Figure 3 shows the number of trees which must be built and searched in order to achieve a fixed recall value. Figure 4 shows the number of distance computations required to search some number of trees.

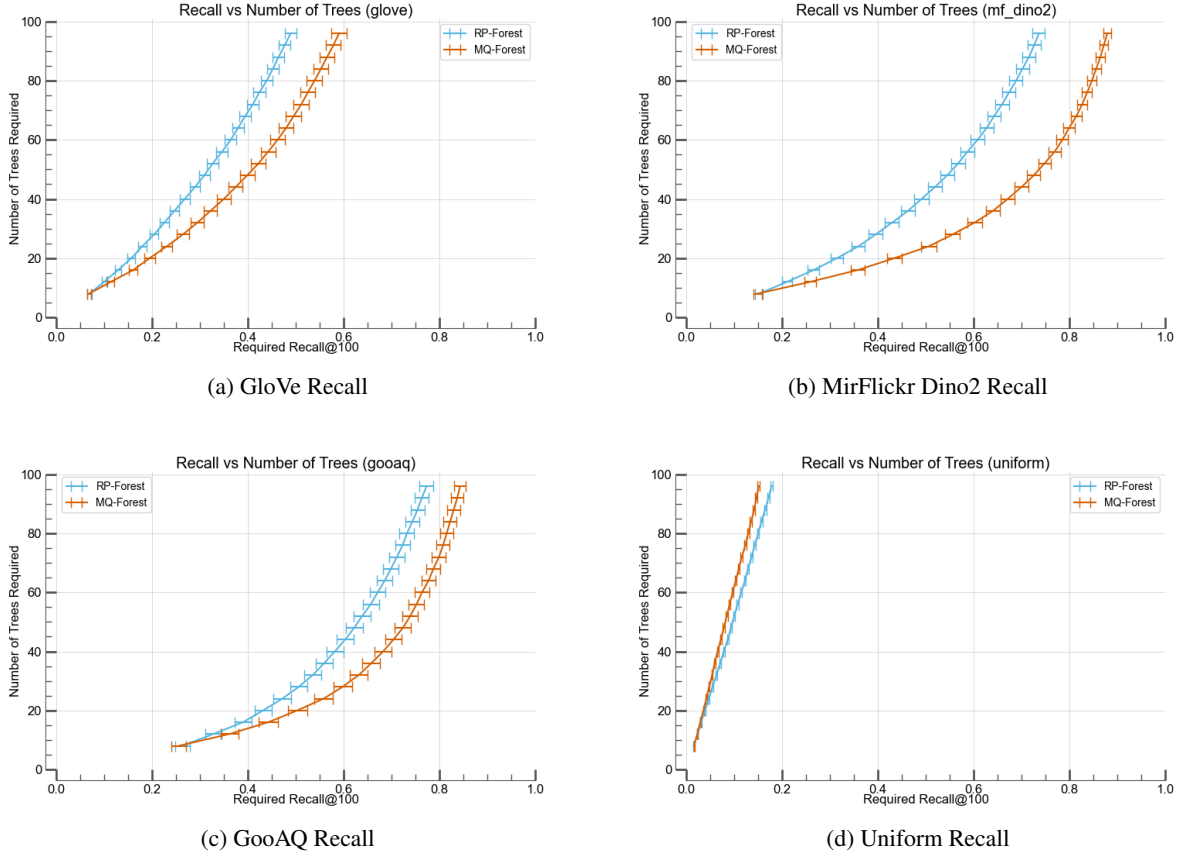


Figure 3: Recall results for an RP-Forest and MQ-Forest consisting of y trees. Note that the degree of vertical difference between the two curves represents improvement; lower is better. For example, MirFlickr can be searched at a recall of approximately 75% in 96 trees for RP-Forest, and approximately 43 for MQ-Forest whilst reducing the number of distance calculations.

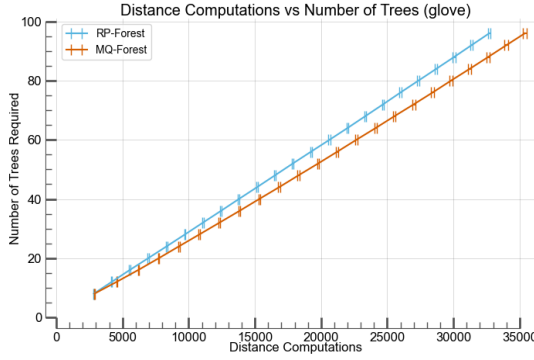
6.2.1 Discussion

We find that, for all neural-network-derived datasets, the MQ-Forest achieves a higher average recall whilst requiring approximately the same number of distance computations to serve a query as RP-Forest with a fixed number of trees.

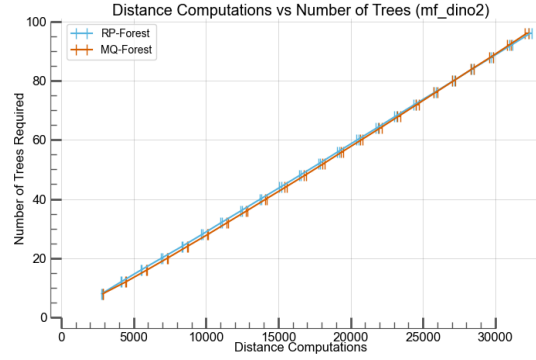
From inspection of Figure 3, it can be seen that RP-Forest must build and search up to 78% more trees to achieve a fixed recall result than MQ-Forest, depending on the dataset. Thus, MQ-Forest can expect expedited build and query times comparatively.

For example, for the GloVe dataset, RP-Forest achieves a recall of 0.49 after 96 trees have been searched. MQ-Forest requires only 68 trees be built and searched to achieve the same recall, a reduction of 29%. The number of metric calls required for this search are also reduced; 32696 are required for the RP-Forest, compared with 25543 for the MQ-Forest, a 22% decrease. Our choice to use the GloVe dataset as an example here is deliberate, as it benefits the least of all neural-network-derived benchmark datasets from the query modification framework. For all other datasets, the MQ-Forest performs better relatively.

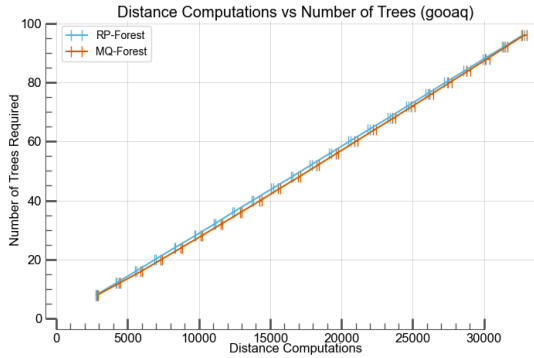
However, for the uniform dataset, the MQ-Forest fails to improve performance. An explanation for poor performance of MQ-Forest in this scenario can be seen in Figure 7. It shows that it is difficult in the context of the uniform dataset to form a good instance of $\langle \hat{c} \rangle$ without a very good instance of $\widehat{knn}(\mathbf{q})$. As RP-Forest does not return a significant number of near neighbours for the uniform dataset, an insufficiently good estimate of $\widehat{knn}(\mathbf{q})$ could not be formed, thus leading MQ-Forest to fail.



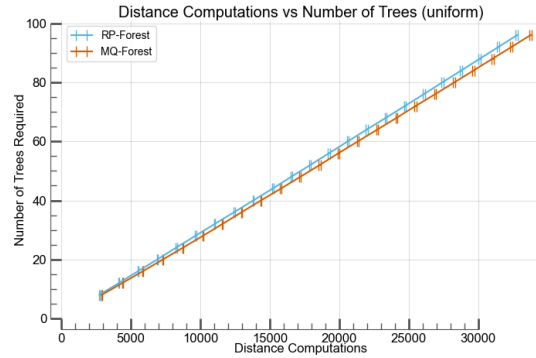
(a) GloVe Distance Computations



(b) MirFlickr Dino2 Distance Computations



(c) GooAQ Distance Computations



(d) Uniform Distance Computations

Figure 4: Distance computations for an RP-Forest and MQ-Forest consisting of y trees.

6.2.2 Fast Production Implementation

We provide a production-ready Cython library for Python, which is available for download from https://github.com/benclaydon/rpforest_mfq/. This version of MQ-Forest is derived from the RP-Forest implementation available from <https://github.com/lyst/rpforest>. In Figure 5, we compare the query throughput of the existing RP-Forest system with our modification for each embedding dataset. We show that our system achieves a higher query throughput for each embedding dataset. We once again emphasise that, as shown in Section 6.2, the MQ-Forest requires significantly less build time than RP-Forest.

6.3 Observations on Distance Between $\mathbf{q}$ and $\langle \mathbf{c} \rangle$

We show that $\mathbf{q}$ and $\langle \mathbf{c} \rangle$ are often distant by computing their distance over a sample of 100 randomly drawn queries from each embedding dataset. The scale of these distances is large enough to cause $\mathbf{q}$ and $\langle \mathbf{c} \rangle$ to frequently take different hash values. As an example, consider the smallest average distance of any dataset, which is 0.33 for the uniform dataset. Using Charikar’s distance to hash collision probability formula [2], we may compute that $\Pr(h(\mathbf{q}) = h(\langle \mathbf{c} \rangle)) = 0.89$ at a distance of 0.33. For some m bit compound hash function, the probability of collision is 0.89^m , which quickly approaches 0 as m grows. Therefore, the set of points retrieved by hashing $\mathbf{q}$ and $\langle \mathbf{c} \rangle$ becomes increasingly likely to differ. However, we have shown that those points retrieved by $\langle \mathbf{c} \rangle$ are expected to be superior.

To aid comparison, we also show in Table 1 the mean distances from $\mathbf{q}$ to its 1nn, 10nn, 100nn. We also record the mean distance to all other points in the dataset. Mean values of these distances are shown in Table 1.

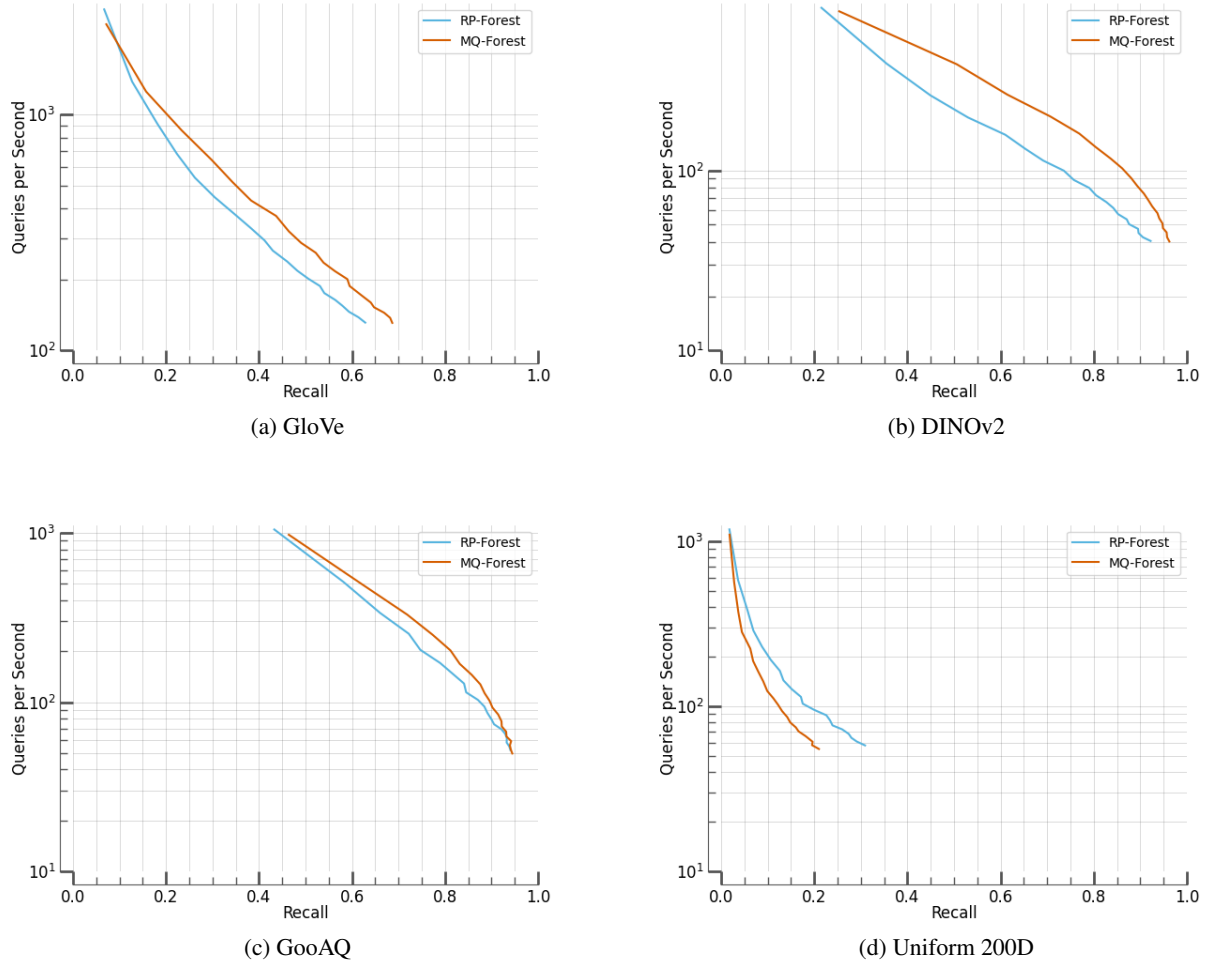


Figure 5: RP-Forest and MQ-Forest: Recall vs Queries Per Second. Note the log-scale y axis. Searching for $k = 20$ near neighbours.

	Glove	MirFlickr	GOOAQ	Uniform 200D
$\ \mathbf{q} - \langle \mathbf{c} \rangle\ _2$	0.59	0.58	0.44	0.33
$\ \mathbf{q} - \mathbf{x}_1\ _2$	0.77	0.71	0.31	1.15
$\ \mathbf{q} - \mathbf{x}_{10}\ _2$	0.84	0.78	0.49	1.18
$\ \mathbf{q} - \mathbf{x}_{100}\ _2$	0.91	0.86	0.69	1.20
Average Interpoint Distance	1.31	1.39	1.38	1.41

Table 1: Mean Euclidean distance from $\mathbf{q}$ to $\langle \mathbf{c} \rangle$ and its k -th nearest neighbour for each embedding dataset and $k = 1, 10, 100$. These distances are average observations measured using 100 randomly sampled vectors from each dataset.

6.4 Eliminating Hash Failure Cases

Here we demonstrate experimentally that hash failure cases are essentially eliminated when $\langle \mathbf{c} \rangle$ is used as input to a binary hash function. To show this we formulate an experiment by initially randomly drawing and excluding a set of 100 queries $Q \subset X$ from each dataset, and a subset $H \subset \mathcal{H}_{rp}$ of 1000 random binary hash functions are drawn.

For all combinations $(h \in H) \times (\mathbf{q} \in Q)$, we measure what fraction of $100nn(\mathbf{q})$ collide with $\mathbf{q}$ and $\langle \mathbf{c} \rangle$. This results in 100,000 individual recall measurements for each dataset. A hash failure is recorded to have occurred at recall values close to 0. We report the distributions of these recall values in Figure 6 for each embedding dataset.

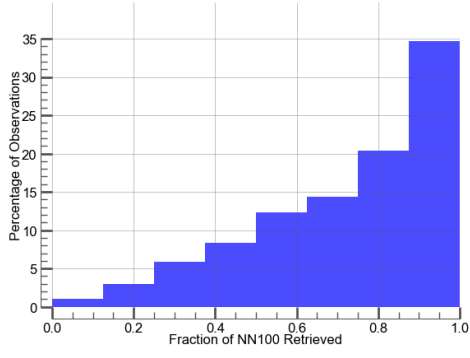
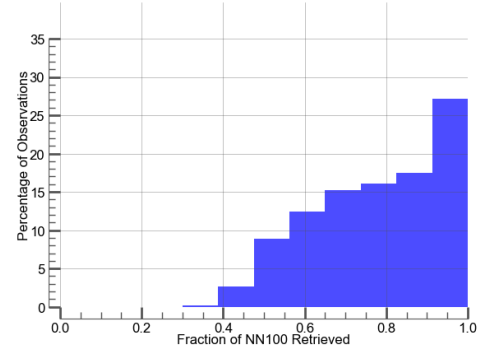
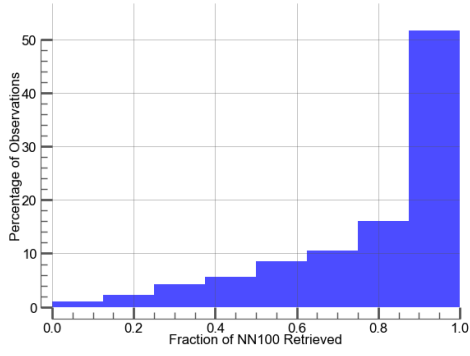
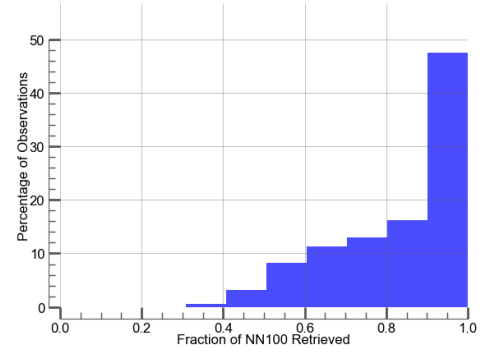
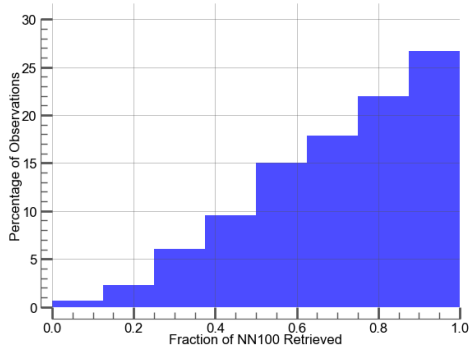
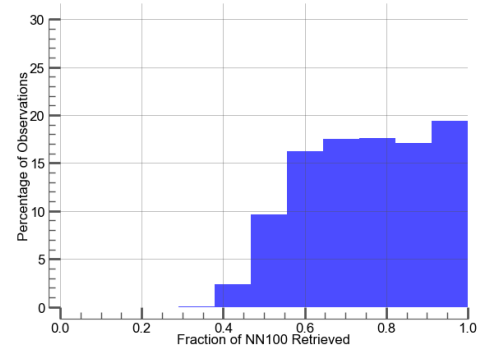
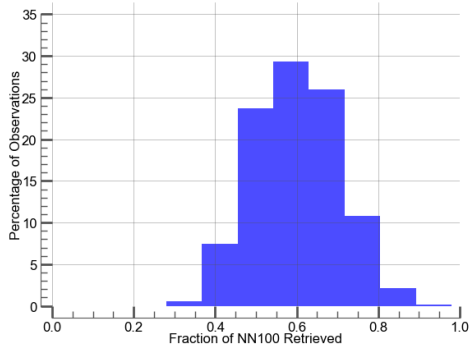
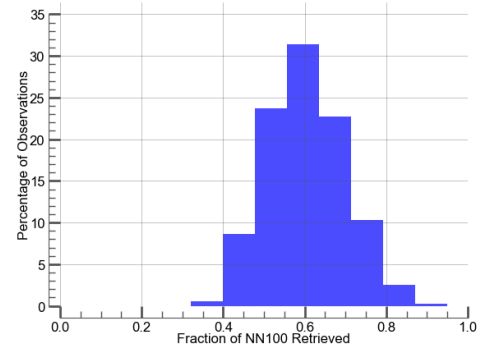
(a) Dino2 (q)(b) Dino2 ($\langle c \rangle$)(c) GooAQ (q)(d) GooAQ ($\langle c \rangle$)(e) GloVe (q)(f) GloVe ($\langle c \rangle$)(g) Uniform (q)(h) Uniform ($\langle c \rangle$)

Figure 6: Fraction of points in $knn(q)$ which collide with q (left column) and $\langle c \rangle$ (right column). Measured using 1000 randomly drawn binary hash functions and 100 queries.

We observe that there are no instances of hash failures when $\langle \mathbf{c} \rangle$ is used as input to randomly drawn binary hash functions for any embedding dataset. This experimentally reinforces the result set out in Section 4.1.1, where the correlation of hash function outputs was explained to be the cause of hash failures. In Appendix D, we further demonstrate that the average value of $\text{Cov}(\mathbf{x} \cdot \mathbf{w}, \mathbf{y} \cdot \mathbf{w} | \mathbf{w} \cdot \mathbf{u} = b)$ for some $\mathbf{x}, \mathbf{y} \in \text{knn}(\mathbf{q})$, some hyperplane normal $\mathbf{w}$, and some constant b is approximately 0 when $\mathbf{u} = \langle \hat{\mathbf{c}} \rangle$.

6.5 Varying Degrees of Approximations of $\widehat{\text{knn}}(\mathbf{q})$

Here we study how well instances of $\langle \hat{\mathbf{c}} \rangle$ perform when derived from sets $\widehat{\text{knn}}(\mathbf{q})$ of various degrees of approximation. If a good instance of $\langle \hat{\mathbf{c}} \rangle$ can be drawn from a weak approximation of $\widehat{\text{knn}}(\mathbf{q})$, then it follows that we may derive a good $\langle \hat{\mathbf{c}} \rangle$ from the result of a coarse search using only a small number of compound hash functions. Initially, we define a quality metric κ to measure the quality of a given $\langle \hat{\mathbf{c}} \rangle$:

$$\kappa \stackrel{\text{def}}{=} \frac{\|\langle \mathbf{c} \rangle - \langle \hat{\mathbf{c}} \rangle\|_2}{\|\langle \mathbf{c} \rangle - \mathbf{q}\|_2}$$

Under this metric, if $\langle \hat{\mathbf{c}} \rangle = \langle \mathbf{c} \rangle$, $\kappa = 0$; this represents the best possible $\langle \hat{\mathbf{c}} \rangle$. If $\langle \hat{\mathbf{c}} \rangle = \mathbf{q}$, then $\kappa = 1$, which represents that $\langle \hat{\mathbf{c}} \rangle$ is expected to have equal performance to the original query. If $0 < \kappa < 1$, it is probable that the instance of $\langle \hat{\mathbf{c}} \rangle$ is a member of Φ and thus will perform favourably to $\mathbf{q}$, but not so well as $\langle \mathbf{c} \rangle$. Any $\langle \hat{\mathbf{c}} \rangle$ with $\kappa > 1$ is worse than the query, and is therefore of no use. Any $\langle \hat{\mathbf{c}} \rangle$ with a κ value of less than 1 is likely to be a member of the set Φ as it is closer to $\langle \mathbf{c} \rangle$ than $\mathbf{q}$ is to $\langle \mathbf{c} \rangle$.

We record κ values of $\langle \hat{\mathbf{c}} \rangle$ instances which were derived from sets containing $k = 100$ elements randomly sampled from the m -near neighbour set. We call such a set Y_m . By increasing m , the set Y_m will contain elements less similar to $\mathbf{q}$ on average.

We form an experiment by randomly drawing and excluding 250 elements from each dataset. Sets Y_m with values of m between 100 and 5000 were then created for each query (apart from queries drawn from the uniform 200D dataset, as the value of κ grows quickly with m for this dataset; therefore we cap $m = 500$ for this dataset). Finally, κ values were measured from each Y_m . Figure 7 shows the average κ values compared with the value of m .

We show that, even when m is significantly larger than k , a useful instance of $\langle \hat{\mathbf{c}} \rangle$ may be derived from the associated set Y_m . An exception to this is the uniform 200D dataset, where the value of m can be at most twice k if a good estimate of $\langle \hat{\mathbf{c}} \rangle$ is to be derived. This property is almost entirely data-dependent as we demonstrate; however, for many neural datasets, a $\langle \hat{\mathbf{c}} \rangle$ of even moderate quality is often much closer to $\hat{\mathbf{c}}$ than $\mathbf{q}$.

Additionally, we show that the average value of κ decreases monotonically as the quality of Y_m improves. In the context of an LSH-based mechanism, the candidate set similarly monotonically improves as new candidates are retrieved and re-ranked. Thus, we can expect the estimate $\langle \hat{\mathbf{c}} \rangle$ in the MQ-Forest to similarly monotonically improve as it is iteratively refined.

7 Conclusion

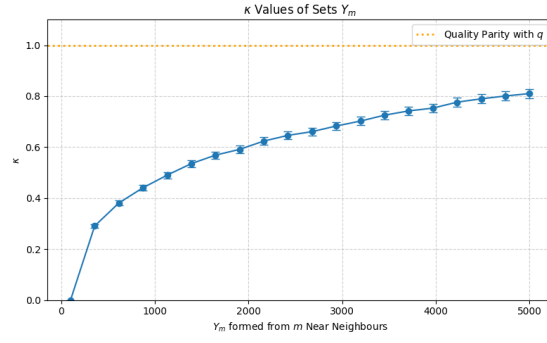
In this paper, we have shown that the build and query procedure of LSH-based ANN mechanisms may be improved via iterative query modification whilst leaving the build procedure entirely unchanged. We show that iterative query modification enjoys a strong statistical and geometric underpinning, as well as positive practical outcomes when applied to a suite of large, high-dimensional benchmark datasets.

8 Resources

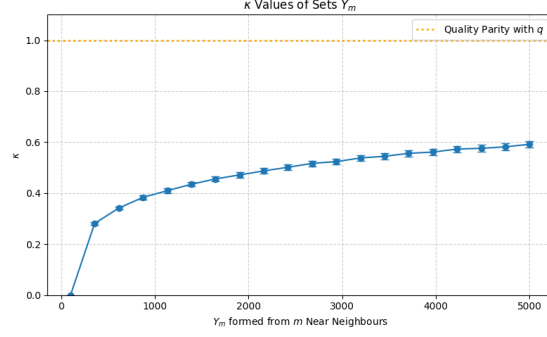
All code used for generating results for this paper can be found at <https://github.com/benclaydon/rp-forest-moving-query/tree/unrestricted-euc>.

References

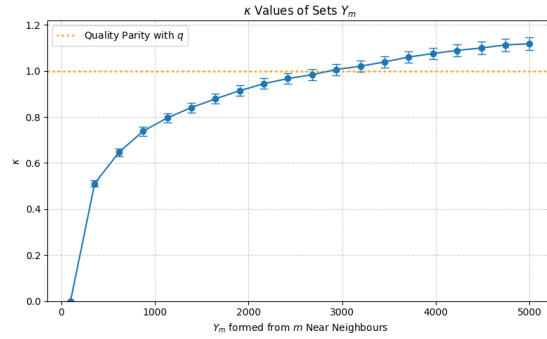
- [1] Alexandr Andoni, Piotr Indyk, Thijs Laarhoven, Ilya Razenshteyn, and Ludwig Schmidt. Practical and optimal lsh for angular distance. (arXiv:1509.02897), 2015. arXiv:1509.02897 [cs].



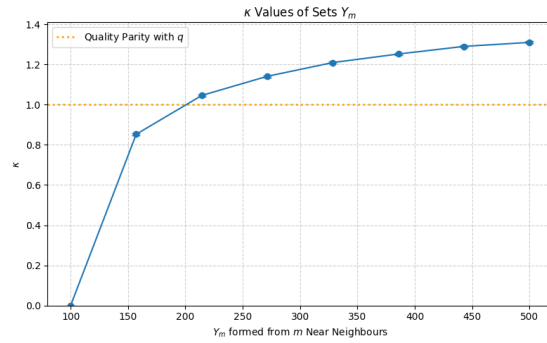
(a) MirFlickr DINOv2



(b) Twitter GloVe



(c) GOOAQ



(d) Uniform 200D

Figure 7: Average value of κ from estimating $\langle c \rangle$ via a random selection of 100 elements from the m nearest neighbours.

- [2] Moses S. Charikar. Similarity estimation techniques from rounding algorithms. In *Proceedings of the thirty-fourth annual ACM symposium on Theory of computing*, STOC '02, page 380–388. Association for Computing Machinery.
- [3] Edgar Chávez, Gonzalo Navarro, Ricardo Baeza-Yates, and José Luis Marroquín. Searching in metric spaces. *ACM Computing Surveys*, 33(3):273–321, 2001.
- [4] Ben Claydon, Richard Connor, and Alan Dearle. Learning to find good hash functions for embeddings. In *Similarity Search and Applications*, page 345–353, Cham, 2026. Springer Nature Switzerland.
- [5] Ben Claydon, Richard Connor, Alan Dearle, and Lucia Vadicamo. Demonstrating the efficacy of polyadic queries. In *Similarity Search and Applications*, page 49–56, Cham, 2025. Springer Nature Switzerland.
- [6] Hui Cui, Fengling Li, Lei Zhu, Jingjing Li, and Zheng Zhang. Online query expansion hashing for efficient image retrieval. *IEEE Transactions on Circuits and Systems for Video Technology*, 34(3):1941–1953, March 2024.
- [7] Sanjoy Dasgupta and Yoav Freund. Random projection trees and low dimensional manifolds. In *Proceedings of the fortieth annual ACM symposium on Theory of computing*, page 537–546, Victoria British Columbia Canada, May 2008. ACM.
- [8] Guglielmo Faggioli, Nicola Ferro, Raffaele Perego, and Nicola Tonellotto. Dimension importance estimation for dense information retrieval. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '24, page 1318–1328, New York, NY, USA, 2024. Association for Computing Machinery.
- [9] Lovedeep Gondara. Random forest with random projection to impute missing gene expression data. In *2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA)*, page 1251–1256, December 2015.
- [10] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. Query-aware locality-sensitive hashing for approximate nearest neighbor search. *Proceedings of the VLDB Endowment*, 9(1):1–12, 2015.
- [11] Mark J. Huiskes and Michael S. Lew. The mir flickr retrieval evaluation. In *Proceedings of the 1st ACM International Conference on Multimedia Information Retrieval*, MIR '08, page 39–43, New York, NY, USA, 2008. Association for Computing Machinery.
- [12] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing - STOC '98*, page 604–613. ACM Press, 1998.
- [13] Daniel Khashabi, Amos Ng, Tushar Khot, Ashish Sabharwal, Hannaneh Hajishirzi, and Chris Callison-Burch. Gooaq: Open question answering with diverse answer types. *arXiv preprint*, 2021.
- [14] Yin-Hsi Kuo, Kuan-Ting Chen, Chien-Hsing Chiang, and Winston H. Hsu. Query expansion for hash-based image object retrieval. In *Proceedings of the 17th ACM international conference on Multimedia*, page 65–74, Beijing China, October 2009. ACM.
- [15] Saar Kuzi, Anna Shtok, and Oren Kurland. Query expansion using word embeddings. In *Proceedings of the 25th ACM International Conference on Information and Knowledge Management*, CIKM '16, page 1929–1932, New York, NY, USA, 2016. Association for Computing Machinery.
- [16] Donghoon Lee, Ming-Hsuan Yang, and Songhwai Oh. Fast and accurate head pose estimation via random projection forests. In *2015 IEEE International Conference on Computer Vision (ICCV)*, page 1958–1966, Santiago, Chile, December 2015. IEEE.
- [17] Donghoon Lee, Ming-Hsuan Yang, and Songhwai Oh. Head and body orientation estimation using convolutional random projection forests. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(1):107–120, January 2019.
- [18] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. Multi-probe lsh: Efficient indexing for high-dimensional similarity search.
- [19] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. (arXiv:1301.3781), 2013. arXiv:1301.3781 [cs].
- [20] Robb J. Muirhead. *Aspects of Multivariate Statistical Theory*. Wiley-Interscience, 2005.
- [21] Mervin E. Muller. A note on a method for generating points uniformly on n-dimensional spheres. *Communications of the ACM*, 2(4):19–20, April 1959.
- [22] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, and et al. Di-nov2: Learning robust visual features without supervision, 2023.

- [23] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In *Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.
- [24] Joseph John Rocchio Jr. Relevance feedback in information retrieval. *The SMART retrieval system: experiments in automatic document processing*, 1971.
- [25] Xu Tan, Jiawei Yang, and Susanto Rahardja. Sparse random projection isolation forest for outlier detection. *Pattern Recognition Letters*, 163:65–73, November 2022.
- [26] Lucia Vadicamo, Francesca Scotti, Alan Dearle, and Richard Connor. Comparative analysis of relevance feedback techniques for image retrieval. In Ichiro Ide, Ioannis Kompatsiaris, Changsheng Xu, Keiji Yanai, Wei-Ta Chu, Naoko Nitta, Michael Riegler, and Toshihiko Yamasaki, editors, *MultiMedia Modeling*, page 206–219, Singapore, 2025. Springer Nature.
- [27] Jinxi Xu and W. Bruce Croft. Improving the effectiveness of information retrieval with local context analysis. *ACM Trans. Inf. Syst.*, 18(1):79–112, January 2000.
- [28] Donghui Yan, Yingjie Wang, Jin Wang, Honggang Wang, and Zhenpeng Li. K-nearest neighbor search by random projection forests. In *2018 IEEE International Conference on Big Data (Big Data)*, page 4775–4781, December 2018. arXiv:1812.11689 [cs, stat].
- [29] Bailing Zhang. Phenotype recognition for rnai screening by random projection forest. *AIP Conference Proceedings*, 1371(1):55–64, 2011.
- [30] Xiang Sean Zhou and Thomas S. Huang. Relevance feedback in image retrieval: A comprehensive review. *Multimedia Systems*, 8(6):536–544, April 2003.

A Description of Datasets

We use four high-dimensional vector datasets of different modalities for experimentation:

1. MirFlickr is a collection of 1 million images submitted by Flickr users [11]. We encode each of these images with the DinoV2S embedder [22], which outputs 384-dimensional embeddings of each image in Euclidean space. The dissimilarity metric for this space is either Euclidean or Cosine distance. Downloadable from <https://zenodo.org/records/15373201>.
2. Twitter GloVe is a set of approximately 1.1 million word embeddings [23]. These embeddings were generated by analysing word co-occurrences from Twitter posts. The dissimilarity metric for this space is either Euclidean or Cosine distance. Downloadable from <https://nlp.stanford.edu/projects/glove/>.
3. GOOAQ is a set of 3 million questions submitted to the Google search engine and their answers [13]. They were embedded using a sentence BERT model to produce 384 dimensional feature vectors. Downloadable at <https://huggingface.co/datasets/sentence-transformers/gooaq>.
4. A uniform spherical dataset of 1 million points in 200 dimensions. Generated using the method described by Muller [21].

B Statistical Model of Hyperplane Hash Function

We derive a statistical model which describes the probability that a point $\mathbf{u} \in \mathbb{R}^d : \|\mathbf{u}\|_2 = 1$ and each member of some set of points $knn(\mathbf{q})$ will hash collide under the family $\mathcal{H}_{rp}$. This class of hash function is parametrised by a random hyperplane $\mathbf{w}$ passing through the origin in a random direction, which can be generated by a vector of numbers drawn from a standard Gaussian distribution. Any points which are at a greater height above $\mathbf{w}$ than a uniformly generated value $a \sim \mathcal{U}[\min(X\mathbf{w}^T), \max(X\mathbf{w}^T)]$ are assigned a value of 1, and all others assigned a value of 0.

To build a model of how this set of hash functions will interact with a point $\mathbf{u}$ and a set $knn(\mathbf{q})$, we begin with the distribution which describes the set of all normals to planes which parametrise all hash functions, as in Equation 5. This is a standard multivariate Gaussian distribution, which is the co-ordinate-wise distribution for generating random hyperplane normals.

$$\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (5)$$

We may condition the distribution of $\mathbf{w}$ given that $\mathbf{u} \cdot \mathbf{w} = b$, where $b \in \mathbb{R}$ represents the height of $\mathbf{u}$ above $\mathbf{w}$. The resultant distribution describes the subset of all planes which both pass through the origin and where $\mathbf{u}$ has a fixed dot product to $\mathbf{w}$, thereby fixing the height of $\mathbf{u}$ above $\mathbf{w}$. A formal derivation of conditional Gaussian distributions can be found in [20].

$$\mathbf{w} \mid \mathbf{u} \cdot \mathbf{w} = b \sim \mathcal{N}(b\mathbf{u}, \mathbf{I} - \mathbf{u}\mathbf{u}^T) \quad (6)$$

Finally, we must model how this distribution of planes interacts with the set $knn(\mathbf{q})$. Consider the set $knn(\mathbf{q})$ as a $k \times d$ matrix $\mathbf{A}$, where the i th row is the i th near neighbour of the original query, $\mathbf{q}$. The matrix multiplication $\mathbf{A}\mathbf{w}$ will produce a vector of the form $[\mathbf{x}_0 \cdot \mathbf{w}, \mathbf{x}_1 \cdot \mathbf{w}, \dots, \mathbf{x}_{k-1} \cdot \mathbf{w}]$ where $\mathbf{x}_i$ is the i th near neighbour.

$$\mathbf{A}\mathbf{w} \mid \mathbf{u} \cdot \mathbf{w} = b \sim \mathcal{N}(\mathbf{A}b\mathbf{u}, \mathbf{A}(\mathbf{I} - \mathbf{u}\mathbf{u}^T)\mathbf{A}^T) \quad (7)$$

$$\sim \mathcal{N}(\mathbf{A}b\mathbf{u}, \mathbf{A}\mathbf{A}^T - (\mathbf{A}\mathbf{u})(\mathbf{A}\mathbf{u})^T) \quad (8)$$

We may alternatively write this distribution in terms of dot products.

$$\mathbf{A}\mathbf{w} \mid \mathbf{u} \cdot \mathbf{w} = b \sim \mathcal{N}(\mu_i = b(\mathbf{u} \cdot \mathbf{x}_i), \Sigma_{ij} = \mathbf{x}_i \cdot \mathbf{x}_j - (\mathbf{x}_i \cdot \mathbf{u})(\mathbf{x}_j \cdot \mathbf{u})), 0 \leq i, j < k \quad (9)$$

This multivariate Gaussian distribution models the mean and covariance of the values $\mathbf{x}_i \cdot \mathbf{w}$ for all $\mathbf{x}_i \in knn(\mathbf{q})$. The hyperplane hash function may be modelled by drawing and thresholding on a random value a , thus creating a binary output:

$$\mathbf{A}\mathbf{w} \mid \mathbf{u} \cdot \mathbf{w} = b \sim \mathcal{N}(\mathbf{A}b\mathbf{u}, \mathbf{A}\mathbf{A}^T - (\mathbf{A}\mathbf{u})(\mathbf{A}\mathbf{u})^T) > a\mathbf{1}_k \quad (10)$$

This models the inner products between $\mathbf{A}$ and $\mathbf{w}$. However, there is not an exact hash collision probability formula for $\mathcal{H}_{rp}$. Instead, our strategy will be to show that the hash collision probability for $\mathcal{H}_{rp}$ is similar to $\mathcal{H}_c$ in Section B.1. We may then consider only the closed-form collision probability of $\mathcal{H}_c$ thereafter. Using this formula, we will show that $\langle \mathbf{c} \rangle$ is a good instance of $\mathbf{u}$ by proving that it maximises the average collision probability, to a first-order approximation.

B.1 Approximating the Effect of a

Consider Equation 10. We may rewrite this expression as a convolution of a normal and 1-dimensional uniform distribution by subtracting a from both sides of the inequality:

$$\mathcal{N}(\mathbf{A}\mathbf{b}\mathbf{u}, \mathbf{A}\mathbf{A}^T - (\mathbf{A}\mathbf{u})(\mathbf{A}\mathbf{u})^T) - a\mathbf{1}_k > 0$$

More precisely, a is distributed as a degenerate k -dimensional uniform distribution, whereby a single, fixed, uniform scalar value is drawn between $\min(X\mathbf{w}^T), \max(X\mathbf{w}^T)$ (where each X is the set of points being hashed, thereby making these bounds the height of the furthest points in X above and below $\mathbf{w}$), which is repeated for all dimensions. We denote this distribution $\mathcal{U}$.

$$\mathcal{N}(\mathbf{A}\mathbf{b}\mathbf{u}, \mathbf{A}\mathbf{A}^T - (\mathbf{A}\mathbf{u})(\mathbf{A}\mathbf{u})^T) - \mathcal{U}(\min(X\mathbf{w}^T), \max(X\mathbf{w}^T)) > 0$$

For ease of notation, we let $W \sim \mathcal{N}(\mathbf{A}\mathbf{b}\mathbf{u}, \mathbf{A}\mathbf{A}^T - (\mathbf{A}\mathbf{u})(\mathbf{A}\mathbf{u})^T)$ and $Y \sim \mathcal{U}(\min(X\mathbf{w}^T), \max(X\mathbf{w}^T))$. We further define a new distribution $Z = W - Y$.

Maximising ACP is equivalent to maximising the probability of the each co-ordinate of Z having the same sign as $b - a$, as the hash of both $\mathbf{u}$ and $\mathbf{x}_i$ will be equal in this case. However, this property is very hard to prove for Z itself, due to the complexity of the distribution². However, we show that finding a $\mathbf{u}$ which maximises the expected value of W also implies that the $\mathbf{u}$ maximises the expected value of Z . Similarly, the $\mathbf{u}$ which minimises the variance of W directly implies it minimises the variance of Z . By showing this, we may work with the much more wieldy X to prove properties of Z , greatly simplifying our derivation.

To achieve this, we must first assume that $\max(X\mathbf{w}^T) \approx -\min(X\mathbf{w}^T)$. This is because the distribution of $X\mathbf{w}^T$ is itself Gaussian, if each member of X is reasonably decorrelated³. We may use this fact to derive a simple mean and variance of the degenerate uniform distribution Y :

$$\mathbb{E}[Y] = 0$$

$$\Sigma[Y] = \frac{\max(\mathbf{x}_i \cdot \mathbf{w})^2}{3} \mathbf{J}_k$$

where $\mathbf{J}_k$ is the $k \times k$ matrix of ones. To compute the mean and variance of the distribution Z , as we assume X and Y are independent (which is reasonable, as the maximum and minimum of a large collection of Gaussian vectors is approximately constant). We can thus rewrite the mean and variance of Z in terms of W and Y :

$$\begin{aligned} \mathbb{E}[Z] &= \mathbb{E}[W] - \mathbb{E}[Y] \\ \implies \mathbb{E}[Z] &= \mathbb{E}[W] \end{aligned}$$

$$\Sigma[Z] = \Sigma[W] + \frac{\max(\mathbf{x}_i \cdot \mathbf{w})^2}{3} \mathbf{J}_k$$

As the expected values of W and Z are equal, and the covariance matrix of Z is simply the covariance matrix of X plus some constant, it follows that $\max(\mathbb{E}[W]) \implies \max(\mathbb{E}[Z])$, and $\min(\text{Var}[W]) \implies \min(\text{Var}[Z])$. Thus we work only with the much simpler distribution:

²The author did derive the distribution of Z , but there is little to be gained from recounting it.

³This should be true, given a correct level of dimensionality reduction has been performed on the data.

$$W = \mathbf{A}\mathbf{w} \mid \mathbf{u} \cdot \mathbf{w} = b \sim \mathcal{N}(\mathbf{A}b\mathbf{u}, \mathbf{A}\mathbf{A}^T - (\mathbf{A}\mathbf{u})(\mathbf{A}\mathbf{u})^T) > 0 \quad (11)$$

Moreover, proving this relationship allows us to simultaneously demonstrate properties of both Charikar’s binary hash function (where $a = 0$), and the hash function used by RP-Forest (which takes a uniform value of a), as the value a has been elided from our analysis. We emphasise that the results derived for $\mathcal{H}_{rp}$ are approximations under the stated independence and symmetry assumptions.

C $\langle \mathbf{c} \rangle$ Increases Mean Collision Probability

We prove that setting $\mathbf{u} = \langle \mathbf{c} \rangle$, to a stated degree of error, maximises the probability of the hashes of $\mathbf{u}$ and some $\mathbf{x}_i \in knn(\mathbf{q})$ colliding. This proof is split into two main parts: Theorem 1 shows that setting $\mathbf{u} = \langle \mathbf{c} \rangle$ maximises the expected value of the distribution derived in Equation 11 if $b > 0$, and minimises it if $b < 0$; Theorem 2 shows that maximising this expected value also maximises the average collision probability to a first order approximation. In Section C.1, we prove that the variance of the distribution in Equation 11 is minimised if $\mathbf{u} = \langle \mathbf{c} \rangle$. This pair of properties is sufficient to motivate that $\langle \mathbf{c} \rangle$ is highly likely to hash collide with members of $knn(\mathbf{q})$.

Theorem 1. *If $\mathbf{u} = \langle \mathbf{c} \rangle$, $\mathbb{E}[\sum_i (\mathbf{w} \cdot \mathbf{x}_i \mid \mathbf{u} \cdot \mathbf{w} = b)]$ is maximised if $b > 0$, and minimised if $b < 0$.*

Proof. As our domain of interest is ℓ_2 -normed spaces, we may rewrite the expectation of Equation 9 using the polarisation identity:

$$b \cdot \begin{pmatrix} 1 - \frac{1}{2} \|\mathbf{u} - \mathbf{x}_1\|_2^2 \\ 1 - \frac{1}{2} \|\mathbf{u} - \mathbf{x}_2\|_2^2 \\ \vdots \\ 1 - \frac{1}{2} \|\mathbf{u} - \mathbf{x}_k\|_2^2 \end{pmatrix}$$

To maximise the sum of all elements, we may show that:

$$\begin{aligned} & \operatorname{argmax}_{\|\mathbf{u}\|_2=1} 1 - \frac{1}{2} \sum_{i=0}^{k-1} \|\mathbf{u} - \mathbf{x}_i\|_2^2 \\ &= \operatorname{argmin}_{\|\mathbf{u}\|_2=1} \sum_{i=0}^{k-1} \|\mathbf{u} - \mathbf{x}_i\|_2^2 \\ &= \operatorname{argmin}_{\|\mathbf{u}\|_2=1} \sum_{i=0}^{k-1} 2 - 2(\mathbf{u} \cdot \mathbf{x}_i) \\ &= \operatorname{argmin}_{\|\mathbf{u}\|_2=1} 2k - 2 \sum_{i=0}^{k-1} (\mathbf{u} \cdot \mathbf{x}_i) \\ &= \operatorname{argmin}_{\|\mathbf{u}\|_2=1} 2k - 2\mathbf{u} \cdot \sum_{i=0}^{k-1} \mathbf{x}_i \\ &= \operatorname{argmax}_{\|\mathbf{u}\|_2=1} \mathbf{u} \cdot \sum_{i=0}^{k-1} \mathbf{x}_i \end{aligned}$$

We must now choose the vector $\mathbf{u}$ which maximises this inner product under restriction that $\|\mathbf{u}\|_2 = 1$. To begin, we apply the Cauchy-Schwarz inequality:

$$\begin{aligned} \mathbf{u} \cdot \sum_{i=0}^{k-1} \mathbf{x}_i &\leq \|\mathbf{u}\|_2 \left\| \sum_{i=0}^{k-1} \mathbf{x}_i \right\|_2 \\ \mathbf{u} \cdot \sum_{i=0}^{k-1} \mathbf{x}_i &\leq \left\| \sum_{i=0}^{k-1} \mathbf{x}_i \right\|_2 \end{aligned}$$

Thus, the maximum of the objective function $\max_{\|\mathbf{u}\|_2=1} \mathbf{u} \cdot \sum_{i=0}^{k-1} \mathbf{x}_i$, for the best possible $\mathbf{u}$, is $\|\sum_{i=0}^{k-1} \mathbf{x}_i\|_2$. Substituting $\mathbf{u} = \langle \mathbf{c} \rangle$ into the objective function, we show that this point attains that maximum value:

$$\begin{aligned}
&= \frac{\mathbf{c}}{\|\mathbf{c}\|_2} \cdot \sum_{i=0}^{k-1} \mathbf{x}_i \\
&= \frac{\sum_{i=0}^{k-1} \mathbf{x}_i}{k \|\frac{1}{k} \sum_{i=0}^{k-1} \mathbf{x}_i\|_2} \cdot \sum_{i=0}^{k-1} \mathbf{x}_i \\
&= \frac{\sum_{i=0}^{k-1} \mathbf{x}_i}{\|\sum_{i=0}^{k-1} \mathbf{x}_i\|_2} \cdot \sum_{i=0}^{k-1} \mathbf{x}_i \\
&= \frac{(\sum_{i=0}^{k-1} \mathbf{x}_i) \cdot (\sum_{i=0}^{k-1} \mathbf{x}_i)}{\|\sum_{i=0}^{k-1} \mathbf{x}_i\|_2} \\
&= \frac{\|\sum_{i=0}^{k-1} \mathbf{x}_i\|_2^2}{\|\sum_{i=0}^{k-1} \mathbf{x}_i\|_2} \\
&= \|\sum_{i=0}^{k-1} \mathbf{x}_i\|_2
\end{aligned}$$

and thus $\langle \mathbf{c} \rangle$ is the ℓ_2 normalised vector which maximises the expected value of Equation 9 as required. $\square$

Theorem 2. $\langle \mathbf{c} \rangle$ is the unit vector which maximises the first-order approximation of the average collision probability.

Proof. The collision probability between any two ℓ_2 -normalised vectors, for Charikar's hash function family $\mathcal{H}_c[2]$, is:

$$\Pr(h(\mathbf{a}) = h(\mathbf{b})) = 1 - \frac{\cos^{-1}(\mathbf{a} \cdot \mathbf{b})}{\pi}$$

Therefore, to maximise the average collision probability, we must maximise the objective function:

$$\operatorname{argmax}_{\|\mathbf{u}\|_2=1} \frac{1}{k} \sum_{i=0}^{k-1} 1 - \frac{\cos^{-1}(\mathbf{x}_i \cdot \mathbf{u})}{\pi}$$

We approximate the function $\cos^{-1}$ with its first-order Maclaurin expansion.

$$= \operatorname{argmax}_{\|\mathbf{u}\|_2=1} \frac{1}{k} \sum_{i=0}^{k-1} 1 - \frac{\frac{\pi}{2} - \mathbf{x}_i \cdot \mathbf{u} + \mathcal{O}((\mathbf{x}_i \cdot \mathbf{u})^3)}{\pi}$$

Simplifying:

$$= \operatorname{argmax}_{\|\mathbf{u}\|_2=1} \frac{1}{2} + \frac{1}{k\pi} \sum_{i=0}^{k-1} \mathbf{x}_i \cdot \mathbf{u} + \mathcal{O}\left(\frac{1}{k} \sum_{i=0}^{k-1} (\mathbf{x}_i \cdot \mathbf{u})^3\right)$$

By Theorem 1, we have already shown that the term $\sum_{i=0}^{k-1} \mathbf{x}_i \cdot \mathbf{u}$ is maximised when $\mathbf{u} = \langle \mathbf{c} \rangle$. Therefore, using $\langle \mathbf{c} \rangle$ maximises the first-order approximation:

$$\langle \mathbf{c} \rangle = \operatorname{argmax}_{\|\mathbf{u}\|_2=1} \frac{1}{2} + \frac{1}{k\pi} \sum_{i=0}^{k-1} \mathbf{x}_i \cdot \mathbf{u}$$

$\square$

C.0.1 Discussion

Upon first inspection, it may seem that a first-order approximation is insufficient to model the function $\cos^{-1}$. However, it is accurate for lower values of the range of the function $[0, 1]$. This is demonstrated by Figure 8 which shows the exact collision probability of two vectors with given inner product, and the first order approximation of this probability.

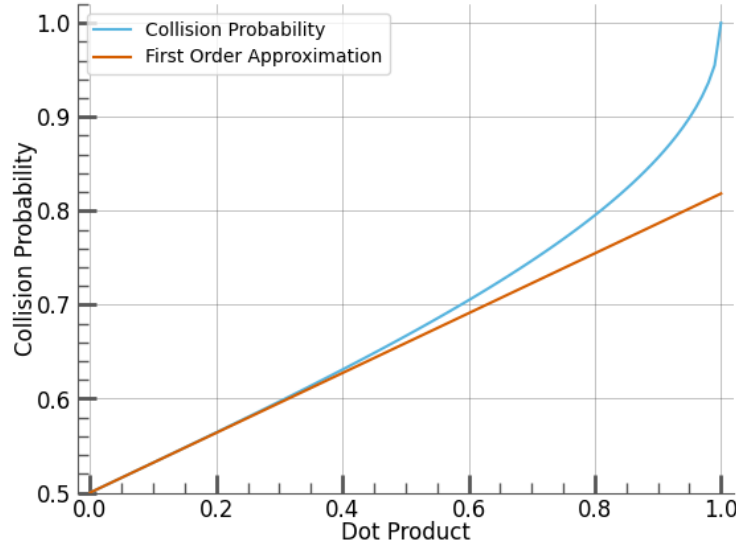


Figure 8: The hash collision probability (y axis) of two ℓ_2 -normalised vectors with a given inner product (x axis).

From inspection, the first order approximation has little error as $\mathbf{x}_i \cdot \mathbf{u}$ is small. As $\mathbf{x}_i \cdot \mathbf{u} \rightarrow 1$, the error increases. However, there are two reasons that this increased error does not trouble us:

1. If there exists a point $\mathbf{v}$ which is very close to $\mathbf{u}$, then its true collision probability will be underestimated by the first-order approximation. Therefore, it will not feature in the sum as prominently as it should. However, their inner product will still be large even when this error is accounted for, and therefore it is likely that $\mathbf{v}$ will quickly be retrieved.
2. Definitionally, the distribution of distances between points in a high-dimensional space has a high mean and low variance. Therefore, we expect few examples of high $\mathbf{u} \cdot \mathbf{v}$ [3] in practice.

C.1 $\langle \mathbf{c} \rangle$ Minimises Average Covariance

Here, we use the matrix Σ as a proxy through which to analyse the output correlation binary hash functions as applied to a fixed pair of points. For a fixed pair of points $\mathbf{x}_i, \mathbf{x}_j$, a matrix entry $\Sigma_{i,j}$ value of -1 represents that, for all $h \in \mathcal{H}_c$, $h(\mathbf{x}_i) \neq h(\mathbf{x}_j)$, and a correlation of $+1$ represents the converse. A score of 0 implies that the outcomes of the hash functions are decorrelated, and that the value $h(\mathbf{x}_i)$ provides no information on the value $h(\mathbf{x}_j)$. Positive covariance matrix entries imply a positive correlation of hash function outcomes, and negative covariance matrix entries imply a negative outcome. However, the relationship between the value of $\Sigma_{i,j}$ and the observed $\text{Cov}(h(\mathbf{x}_i), h(\mathbf{x}_j))$ may not be linear.

We define the *average correlation* more formally to be the mean value of $\Sigma_{i,j}$ for all i, j . If the average correlation is high, then it is likely that many or all members of $\text{knn}(\mathbf{q})$ will have equal hash function output if a single function from $\mathcal{H}_c$ is applied. However, if $\mathbf{q}$ produces the opposite output, then hash failure occurs. To minimise the probability of hash failures, a $\mathbf{u}$ must be chosen which induces a low average correlation.

We show that using $\mathbf{u} = \langle \mathbf{c} \rangle$ minimises the average correlation:

Proof.

$$\begin{aligned}
& \operatorname{argmin}_{\|\mathbf{u}\|_2=1} \frac{1}{k^2} \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} \mathbf{x}_i \cdot \mathbf{x}_j - (\mathbf{x}_i \cdot \mathbf{u})(\mathbf{x}_j \cdot \mathbf{u}) \\
& \text{Because } \mathbf{x}_i, \mathbf{x}_j \text{ are constant} \\
& = \operatorname{argmax}_{\|\mathbf{u}\|_2=1} \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} (\mathbf{x}_i \cdot \mathbf{u})(\mathbf{x}_j \cdot \mathbf{u}) \\
& = \operatorname{argmax}_{\|\mathbf{u}\|_2=1} \left(\sum_{i=0}^{k-1} (\mathbf{x}_i \cdot \mathbf{u}) \right) \left(\sum_{j=0}^{k-1} (\mathbf{x}_j \cdot \mathbf{u}) \right) \\
& = \operatorname{argmax}_{\|\mathbf{u}\|_2=1} \left(\sum_{i=0}^{k-1} \mathbf{x}_i \cdot \mathbf{u} \right)^2
\end{aligned}$$

□

As we showed in Theorem 1, $\langle \mathbf{c} \rangle$ is the point which maximises the term $\sum_{i=0}^{k-1} \mathbf{x}_i \cdot \mathbf{u}$. Therefore, it also maximises $(\sum_{i=0}^{k-1} \mathbf{x}_i \cdot \mathbf{u})^2$. As Σ is a proxy for describing how correlated hash functions are when applied to pairs of points, we expect that hash function correlation is similarly low as well.

D Distribution of Off-Diagonal Covariance Values

We demonstrate that using $\mathbf{u} = \langle \mathbf{c} \rangle$ causes the *average* off-diagonal entry of the covariance matrix to be approximately 0. We measure covariance matrices for a large set of queries and record their distribution. We formulate this experiment as follows:

1. For a given query, generate 1000 random hyperplanes such that $|\mathbf{u} \cdot \mathbf{w}| < 0.005$ (approximating planes where $b = 0$). Note that the choice of $b = 0$ is arbitrary as the value of b does not influence the covariance as per Equation 9, but must be fixed such that the mean value is stable.
2. For each plane $\mathbf{w}$, we compute the values $A\mathbf{w}$, or the height of each near neighbour above $\mathbf{w}$. We store these results in D , a 1000×100 matrix representing the heights of the 100 nearest neighbours to the query over all 1000 hyperplanes.
3. Finally, we measure the covariance between the columns of the matrix D . This measures the degree of correlation between $\mathbf{x}_i \cdot \mathbf{w}$ and $\mathbf{x}_j \cdot \mathbf{w}$ over a large sample of planes. Recall that these covariances are bounded between ± 1 .

Figure 9 shows the distribution of all such covariances aggregated over all queries. For all datasets, setting $\mathbf{u} = \langle \mathbf{c} \rangle$ reduces the average covariance to approximately zero.

E Frequency of Candidate Set Updates

Here we measure how often RP-Forest adds points to its candidate set after the i th tree is searched, thereby estimating the rate at which the candidate set changes. If this value is below $\frac{k}{2}$, then the fast query centroid algorithm set out in Section 5.3 can be used to decrease the cost of computing $\langle \hat{\mathbf{c}} \rangle$.

To measure this property, we formulate an experiment whereby an RP-Forest formed of 19 trees is constructed from each dataset. We then search the constructed forests using 100 queries drawn and excluded from each dataset at random, before each forest was constructed. After each tree in the forest is searched, we measure the number of elements which were added to the candidate set. We report this metric as $\Delta - kNN$. We show the average $\Delta - kNN$ value after each tree is searched for all datasets in Figure 10.

We show that, on average, points are added to the candidate set infrequently. Critically, we show that no dataset adds points to the candidate set at a rate of above $\frac{k}{2}$ per tree beyond the first. Therefore the fast centroid computation algorithm (described in Section 5.3) can be used to decrease the cost of computing $\langle \hat{\mathbf{c}} \rangle$.

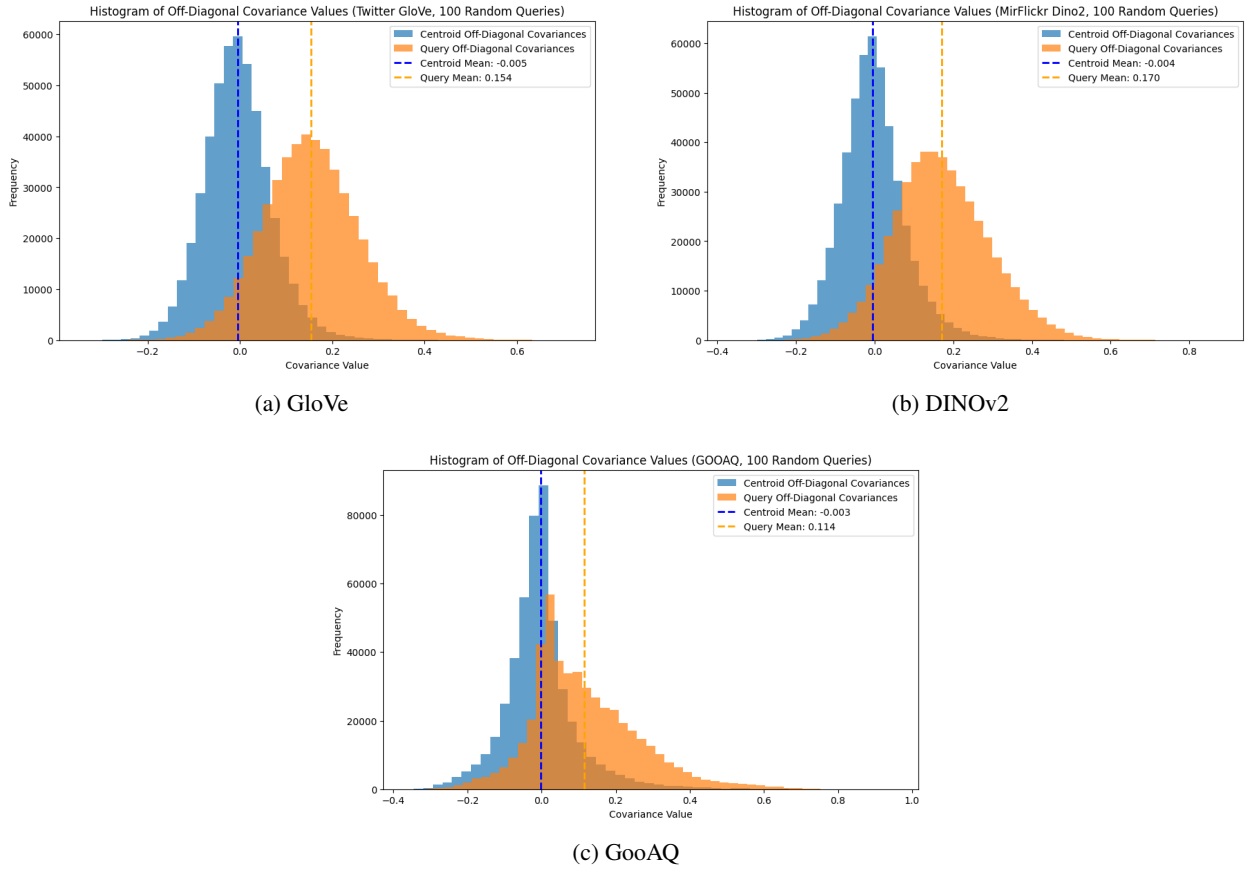


Figure 9: Aggregated off-diagonal covariance values when applying a large number of binary hash functions to $\langle \mathbf{c} \rangle$ (blue) and $\mathbf{q}$ (orange) as instances of $\mathbf{u}$. Measured over 100 random queries.

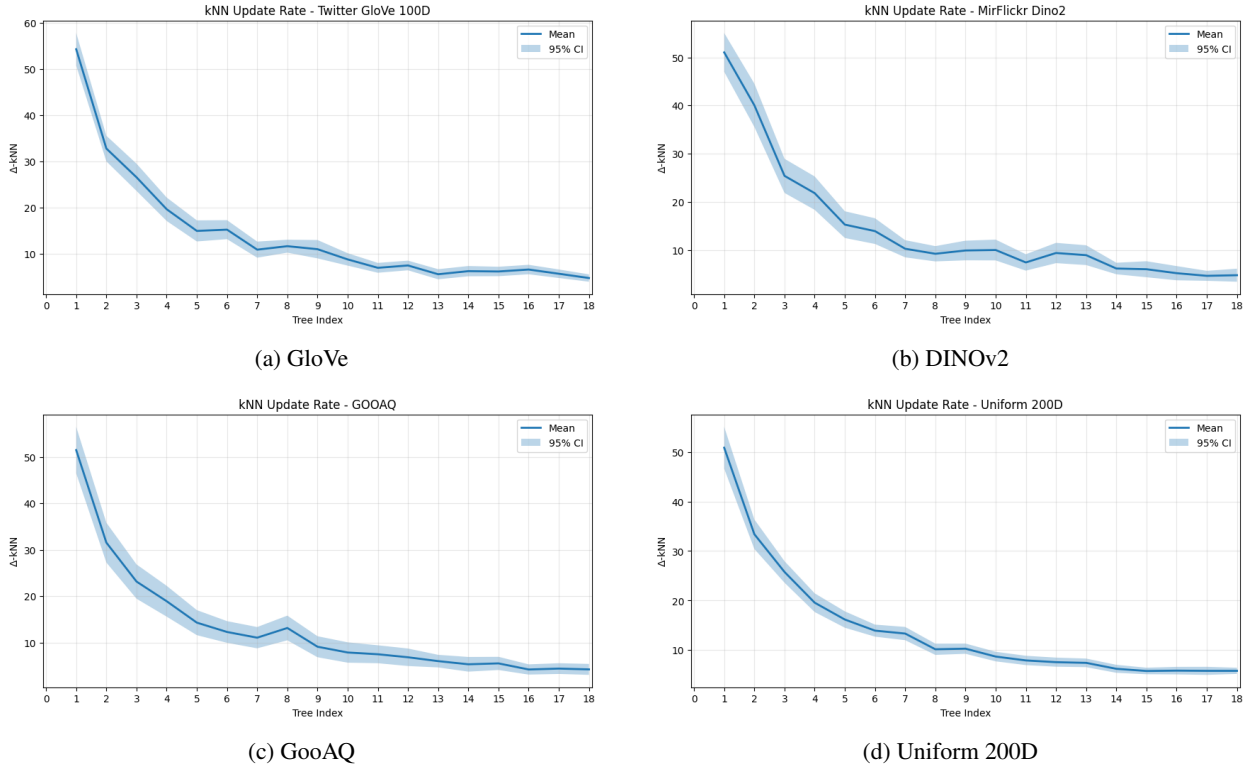


Figure 10: Rate of change of the candidate set after trees in an RP-Forest are searched. $k = 100$.